

Rajalakshmi Engineering College

Name: SHREYA KS
Email: 240701506@rajalakshmi.edu.in
Roll no: 240701506
Phone: 9789293683
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, InvalidAmountException and InsufficientFundsException, both extending the Exception class. Throw an InvalidAmountException with a message if the deposit amount is less than or equal to zero. Throw an InsufficientFundsException if the withdrawal amount is greater than the available balance. Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value B, representing the initial balance.

The second line consists of a double value D, representing the deposit amount.

The third line consists of a double value W, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an `InvalidAmountException` occurs, print "Error: [D] is not valid".

If an `InsufficientFundsException` occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1

270.2

150.3

Output: Amount Withdrawn: 150.3

Current Balance: 1170.0

Answer

```
import java.util.*;
```

```
class InvalidAmountException extends Exception {  
    public InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

```
class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double balance = sc.nextDouble();
        double deposit = sc.nextDouble();
        double withdraw = sc.nextDouble();

        try {
            if (deposit <= 0) {
                throw new InvalidAmountException(deposit + " is not valid");
            }
            balance += deposit;
            if (withdraw > balance) {
                throw new InsufficientFundsException("Insufficient funds");
            }
            balance -= withdraw;
            System.out.printf("Amount Withdrawn: %.1f Current Balance: %.1f",
withdraw, balance);
        } catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
        }
        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Theo is trying to update his payment information on a subscription-based streaming service. To proceed, the system requires Theo to provide a valid credit card number consisting of 16 digits. However, Theo wants to make sure that the credit card number he enters meets the specified criteria with proper exception handling.

The credit card number must consist of exactly 16 digits. If the entered credit card number does not meet the specified criteria, the program should throw a custom exception, `InvalidCreditCardException`, and provide Theo with specific error messages: If the length of the credit card number is not 16 digits, the exception message should be: "Invalid credit card number length." If the credit card number contains non-numeric characters, the exception message should be: "Invalid credit card number format."

Implement a custom exception, `InvalidCreditCardException`, to fulfill Theo's requirements and keep his payment information secure.

Input Format

The input consists of a string value 's', consisting of the 16-digit credit card number.

Output Format

The output is displayed in the following format:

If the entered credit card number is valid, the program should output a success message:

"Payment information updated successfully!"

If the entered credit card has more than 16 digits or less than 16 digits it displays

"Error: Invalid credit card number length."

If the entered 16-digit credit card has non-integers it displays

"Error: Invalid credit card number format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1234567890123456

Output: Payment information updated successfully!

Answer

```
import java.util.Scanner;

class InvalidCreditCardException extends Exception {
    public InvalidCreditCardException(String message) {
        super(message);
    }
}

class CreditCardValidator {
    public static void validateCreditCardNumber(String cardNumber) throws
InvalidCreditCardException {
        if (cardNumber.length() != 16) {
            throw new InvalidCreditCardException("Error: Invalid credit card number
length.");
        }
        if (!cardNumber.matches("\\d{16}")) {
            throw new InvalidCreditCardException("Error: Invalid credit card number
format.");
        }
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    String cardNumber = scanner.nextLine();
    scanner.close();

    try {
        validateCreditCardNumber(cardNumber);
        System.out.println("Payment information updated successfully!");
    } catch (InvalidCreditCardException e) {
        System.out.println(e.getMessage());
    }
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Faustus is managing his bank account and wants to create a program to

update his account balance based on certain conditions. However, he needs to handle specific scenarios related to invalid inputs and insufficient balances. Faustus wants to update his account balance. He inputs the current balance and the amount to be updated.

The initial account balance should be positive. If Faustus enters a negative initial balance, the program should throw an `InvalidAmountException` with the message "Invalid amount. Please enter a positive initial balance." If the amount to be updated is negative, the program should check if the subtraction results in a negative balance. If so, it should throw an `InsufficientBalanceException` with the message "Insufficient balance." If the amount to be updated is positive, it should be added to the current balance, and the new balance should be printed.

Implement a custom exception, `InvalidAmountException`, and `InsufficientBalanceException`, to manage his bank account.

Input Format

The first line of input consists of a double value 'd', representing the initial account balance.

The second line of input consists of a double value 'd1', representing the amount to be updated.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Account balance updated successfully! New balance: {new_balance}"

where {new_balance} is the updated account balance.

If the initial bank amount is negative it displays

"Error: Invalid amount. Please enter a positive initial balance."

If the updated amount exceeds the initial account balance in withdrawal it displays

"Error: Insufficient balance."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1000
500

Output: Account balance updated successfully! New balance: 1500.0

Answer

```
import java.util.*;

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double initialBalance = sc.nextDouble();
        double updateAmount = sc.nextDouble();
        try {
            if (initialBalance < 0) {
                throw new InvalidAmountException("Invalid amount. Please enter a positive initial balance.");
            }
            if (updateAmount < 0) {
                if (initialBalance + updateAmount < 0) {
                    throw new InsufficientBalanceException("Insufficient balance.");
                } else {
                    initialBalance += updateAmount;
                    System.out.println("Account balance updated successfully! New
```

```

        balance: " + initialBalance);
    }
    } else {
        initialBalance += updateAmount;
        System.out.println("Account balance updated successfully! New
balance: " + initialBalance);
    }
    } catch (InvalidAmountException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (InsufficientBalanceException e) {
        System.out.println("Error: " + e.getMessage());
    }
    }
    sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A company is developing a user registration system that requires users to provide valid email addresses. The development team is implementing an EmailValidator program to ensure that the entered email addresses meet certain criteria using exception handling.

The email address must contain the "@" symbol. The email address must consist of a non-empty username (before "@" symbol) and a non-empty domain (after "@" symbol). The domain part of the email address must contain at least one period ("."). The email address must not contain leading or trailing spaces.

Implement a custom exception, InvalidEmailException, to fulfill the company's requirements and validate it according to the specified rules.

Input Format

The input consists of a string value 's', which represents the email address.

Output Format

The output is displayed in the following format:

If the entered email address is valid according to the specified rules, the program prints:

If the entered email address misses the username or domain part or misses "@" symbol or has two or more "@" symbols or misses '.' in the domain part it outputs:

Refer to the sample output for formatting specifications.

Input: johndoe@example.com

Answer

```
class InvalidEmailException extends Exception {
    public InvalidEmailException(String message) {
        super(message);
    }
}
```

```
int atCount = email.length() - email.replace("@", "").length();
if (atCount != 1) {
    throw new InvalidEmailException("Error: Invalid email format.");
}
```

```
String[] parts = email.split("@");  
if (parts.length != 2 || parts[0].isEmpty() || parts[1].isEmpty()) {  
    throw new InvalidEmailException("Error: Invalid email format.");  
}
```

```
String domainPart = parts[1];
```

```
if (!domainPart.contains(".")) {  
    throw new InvalidEmailException("Error: Invalid email format.");  
}
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    String email = scanner.nextLine();  
    scanner.close();
```

```
    try {  
        validateEmail(email);  
        System.out.println("Email address is valid!");  
    } catch (InvalidEmailException e) {  
        System.out.println(e.getMessage());  
    }
```

Status : Correct

Marks : 10/10